

Análisis ORB SLAM en FPGA

1. Objetivos

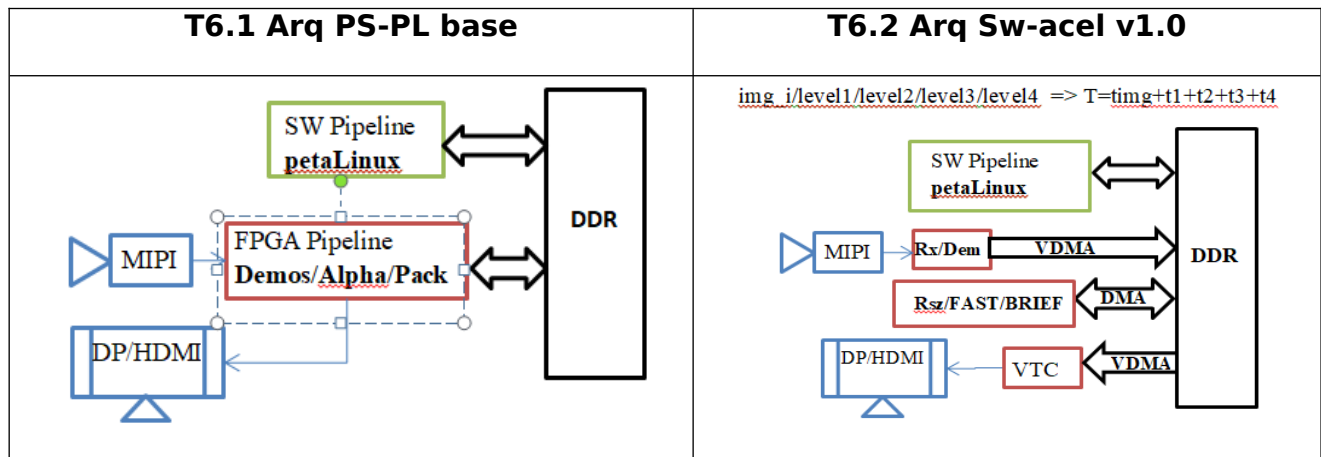
OBJ1. Estimar los recursos necesarios para implementación de la arquitectura correspondiente a la tarea 6.2 de la planificación

(<https://github.com/embention/MissionComputer/issues/98>)

OBJ2. Seleccionar una placa con capacidad suficiente para realizar el desarrollo y el test previo a la integración en el hardware del MissionComputer

OBJ3. Diseñar la arquitectura Sw-acel v1.0

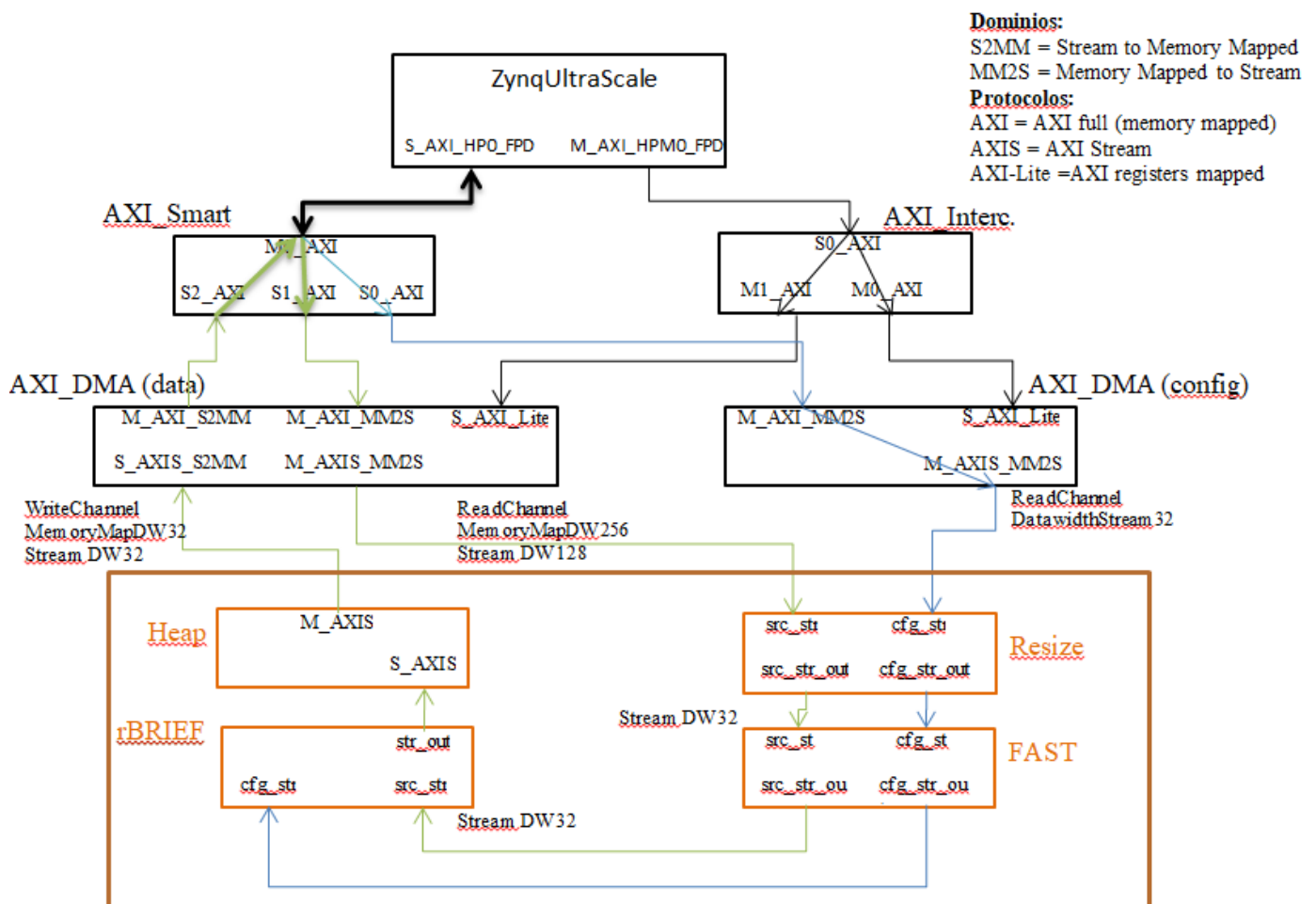
PRODUCTO: VERONTE GNSS DENIED																													
1ª versión - PRODUCTO MÍNIMO VIABLE																													
		2023																											
		OCTUBRE					NOVIEMBRE					DICIEMBRE					ENERO					FEBRERO					MARZO		
TAREA	RESPONSABLE	S40	S41	S42	S43	S44	S45	S46	S47	S48	S49	S50	S51	S52	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S1	S2	S3
1 Prototipo pipeline SW OK	JOHN	★	OK																										
2 Pruebas en vuelo con Jetson	JOHN/AVIONICS/PDI													★															
3 1ª versión SW en Zynq Ultrascale+ (ZUS+)	VICTOR/JOHN																												
4 SW en Zynq Ultrascale+ (ZUS+) OK, Petalinux	VICTOR/JOHN																												
5 Análisis aceleración SW en FPGA	SERGIO																												
Subir documento de análisis en issue	SERGIO																												
6 Modificación SW para aceleración en FPGA																													
6.1 Versión 1. Diseño de arquitectura PS-PL																													
Sincronización/Buffering. Testeo pipeline-FPGA	VÍCTOR/JOHN																												
Configurar drivers cámara en Petalinux	VICTOR																												
6.2 Versión 2. SW acelerado 1																													
Diseño de referencia Wang 2021	SERGIO																												
Acelerar Resize/FAST?	SERGIO																												
Testeo pipeline - FPGA	VÍCTOR/JOHN																												
6.3 Versión 3. SW acelerado 2																													
Acelerar FAST/BRIEF	SERGIO																												
Testeo pipeline - FPGA	VÍCTOR/JOHN																												
7 SW acelerado en FPGA	SERGIO/JOHN/VICTOR																												
8 Añadir compatibilidad con Armbian	JOHN/JOHN																												



2. Arquitectura del pipeline ORB (proyecto ac²SLAM)

Está compuesta por 4 etapas:

- Resize: desescalado de la imagen. Máx resolución Nfx1241
- FAST: extractor de características. Ventana 9x9,
- BRIEF: cálculo de los descriptores. Ventana 21x21,
- Heap: ordenación de los descriptores. Genera un máximo de 8.192 descriptores por escala



3. Implementación ORB completo (wang) vs capacidad de pastillas 5eV y 7eV

Recursos vs resultado de github	ORB acSLAM para GNSS	xczu5ev (Genesys/Kria)	xczu7ev (XCU104)
	Vivado2019.1 (github)		

CLB LUTs	174,737	117,200	230,400
CLB FF	93,306	234,240	460,800
RAMB36/RAMB18	285	144	312
URAM	6	64	96
DSPs	180	1,248	1,728

4. Implementación ORB complete con distintas versiones de Vivado

a) ORB acSLAM resize/FAST/Brief/Heap (**vivado2019.1** / HLS 2019.1) en 7EV

Resource	Utilization	Available	Utilization %
LUT	167892	230400	72.87
LUTRAM	2722	101760	2.67
FF	89773	460800	19.48
BRAM	284.50	312	91.19
DSP	180	1728	10.42
BUFG	11	544	2.02

b) ORB acSLAM resize/FAST/Brief/Heap (**vivado2022.1**/ HLS 2019.1) en 7EV

Resource	Utilization	Available	Utilization %
LUT	166279	230400	72.17
LUTRAM	2661	101760	2.61
FF	89739	460800	19.47
BRAM	284.50	312	91.19
DSP	180	1728	10.42
BUFG	11	544	2.02

c) ORB acSLAM resize/FAST/Brief/ (vivado2020.1/ HLS 2020.1) en 7EV

Resource	Utilization	Available	Utilization %
LUT	173873	230400	75.47
LUTRAM	2104	101760	2.07
FF	96182	460800	20.87
BRAM	89	312	28.53
DSP	183	1728	10.59
IO	12	360	3.33
BUFG	19	544	3.49
MMCM	2	8	25.00
PLL	1	16	6.25

5. Recursos por IP (estimación Vitis HLS)

a) Variando la resolución de la imagen

acSLAM	2019.1 (7EV, n=4, full_res)				2022.1 (5EV, n=1, full_res)				2022.1 (5EV, n=1, 640x480)			
IP/ recursos	BRA M	DS P	FF	LUT	BRA M	DS P	FF	LUT	BRA M	DS P	FF	LUT
resize	4	236	7712	34475	4	224	8759	35263	4	192	12686	35241
FAST	18	2	22022	89864	18	2	18075	81599	18	2	18047	81576
RS-BRIEF	69	12	43234	125058	72	7	46933	282955	72	7	46954	282988

(rojo) demasiado altos comparados con la versión 2019, es necesario revisar

Conclusión: Aparentemente sólo el IP resize reduce su ocupación al reducir la resolución

b) Variando el grado de paralelismo (4*n pixels procesados simultáneamente)

acSLAM	2022.1 (5EV, n=4, full_res)				2022.1 (5EV, n=2, full_res)				2022.1 (5EV, n=1, full_res)			
IP/ recursos	BRA M	DSP	FF	LUT	BRA M	DSP	FF	LUT	BRA M	DSP	FF	LUT
resize	4	224	8759	35263	2	120	6986	21350	2	68	6104	15381
FAST	No aplica*				No aplica*				18	2	18075	81599
BRIEF	No aplica*				No aplica*				72	7	46933	282955

(*) FAST y BRIEF, por el tipo de procesamiento que realizan solo pueden implementarse con n=1 (4 pixels en paralelo)

acSLAM	2019.1 (5EV, n=4, full_res)				2019.1 (5EV, n=2, full_res)				2019.1 (5EV, n=1, full_res)			
IP/ recursos	BRA M	DSP	FF	LUT	BRA M	DSP	FF	LUT	BRA M	DSP	FF	LUT
resize	4	236	8296	33180	2	132	5608	19383	-	-	-	-
FAST	No aplica				No aplica				18	2	21999	88483
BRIEF	No aplica				No aplica				69	12	41928	130425

Conclusión: sólo IP resize se ve afectado por el cambio de grado de paralelismo. La reducción es mayor que al cambiar la resolución de imagen

6. Capacidad máxima de la pastilla 5EV para testar el pipeline.

acSLAM	2019.1 (5EV, n=2, 640x480)				2019.1 (5EV, n=1, 640x480)			
IP/ recursos	BRA M	DSP	FF	LUT	BRA M	DSP	FF	LUT
resize	2	111	5518	18556	-	-	-	-
FAST	No aplica				18	2	21976	88455
BRIEF	No aplica				69	12	41908	130350

Implementación 1: Cambio grado de paralelismo y resolución, sólo **las 2 primeras etapas**, implementación con vivado 2022.1 (IPs 2019.1)

Project_3_rsz(n=2,640x480)_FAST(n=1,640x480) Vivado 2022.1 / IPs 2019.1

Resource	Utilization	Available	Utilization %
LUT	50562	117120	43.17
LUTRAM	1897	57600	3.29
FF	31934	234240	13.63
BRAM	17.50	144	12.15
DSP	92	1248	7.37
BUFG	4	352	1.14

Implementación 2: Cambio grado de paralelismo y resolución, sólo **las 3 primeras etapas**, implementación con vivado 2022.1 (IPs 2019.1)

Project_4: el número de LUTs supera la capacidad máxima de la pastilla 5EV

Implementación 3: Cambio grado de paralelismo y resolución, sólo **las 2 primeras etapas + MIPI-DP**, implementación con vivado 2022.1 (IPs 2019.1)

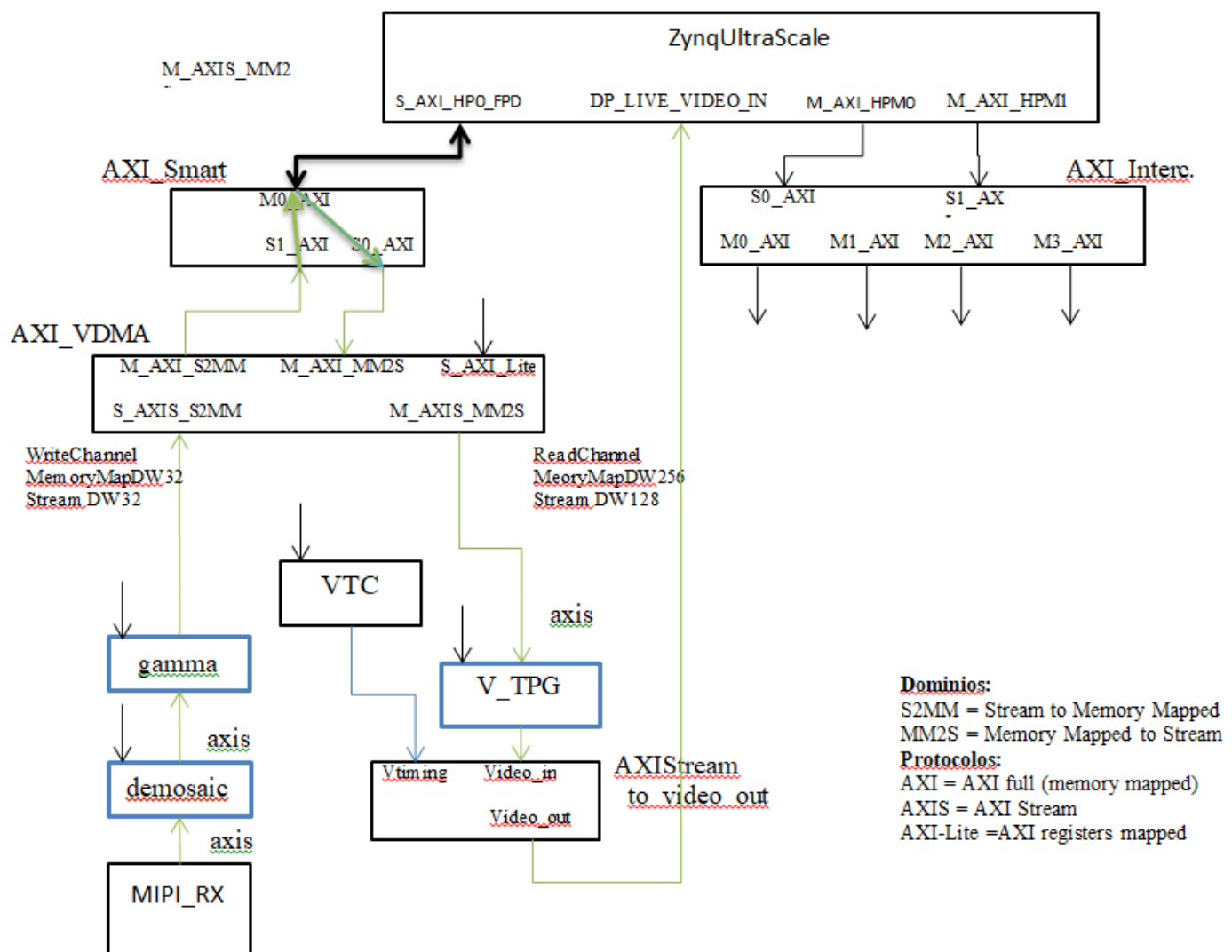
Project_5_ORB-MIPI_n2_640 Vivado 2022.1 / IPs 2019.1

7. Test de rendimiento de distintas configuraciones del pipeline ORB

TODO no crítico

8. Arquitectura MIPI-DP

(https://github.com/gtaylormb/ultra96v2_imx219_to_displayport)



Recursos	MIPI-DP	xczu5ev (Genesys/Kria)	xczu7ev (XCU104)
CLB LUTs	20,940	117,200	230,400
CLB FF	26,405	234,240	460,800
RAMB36/RAMB18	32	144	312
URAM	2	64	96
DSPs	8	1,248	1,728

9. Arquitectura ORB+MIPI-DP

a) Implementación **ORB+MIPI-DP** full, n=4, Heap, **7EV 2020.1**

[DRC UTLZ-1] Resource utilization: RAMB18 and RAMB36/FIFO over-utilized in Top Level Design (This design requires more RAMB18 and RAMB36/FIFO cells than are available in the target device. This design requires **626 of such cell types but only 624 compatible sites are available in the target device**. Please analyze your synthesis results and constraints to ensure the design is mapped to Xilinx primitives as expected. If so, please consider targeting a larger device.)

- b) Implementación, **no Heap (con y sin DP)** full, n=4, Heap, demosing **URAM, 7EV 2020.1**

ERROR

- c) Implementación full, n=4,(sin DP) **no Heap, 7EV 2020.1**

[DRC RTSTAT-6] Partial route conflicts: 16754 net(s) have a partial conflict. The problem bus(es) and/or net(s) are

design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_buf_U0/image_buf_13_V_U/RS_BRIEF_process_buf_image_buf_0_V_ram_U/ADDRBWRADDR[0],
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_buf_U0/image_buf_13_V_U/RS_BRIEF_process_buf_image_buf_0_V_ram_U/ADDRBWRADDR[1],
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_buf_U0/image_buf_13_V_U/RS_BRIEF_process_buf_image_buf_0_V_ram_U/ADDRBWRADDR[2],
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_buf_U0/image_buf_13_V_U/RS_BRIEF_process_buf_image_buf_0_V_ram_U/ADDRBWRADDR[4],
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_buf_U0/image_buf_13_V_U/RS_BRIEF_process_buf_image_buf_0_V_ram_U/ADDRBWRADDR[5],
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_buf_U0/image_buf_13_V_U/RS_BRIEF_process_buf_image_buf_0_V_ram_U/ADDRBWRADDR[6],
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_buf_U0/image_buf_13_V_U/RS_BRIEF_process_buf_image_buf_0_V_ram_U/ADDRBWRADDR[7],
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_buf_U0/image_buf_13_V_U/RS_BRIEF_process_buf_image_buf_0_V_ram_U/ADDRBWRADDR[8],
design_1_i/ps8_0_axi_periph/s00_couplers/auto_ds/inst/gen_downsizer.gen_simple_downsizer.axi_downsizer_inst/USE_READ.read_addr_inst/cmd_queue/inst/fifo_gen_inst/inst_fifo_gen/gconvfifo.rf/grf.rf/rstblk/AR[0],
design_1_i/RS_BRIEF_0/U0/regslice_both_outStream_V_data_V_U/B_V_data_1_load_B,
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_RS_BRIEF_U0/mul_28ns_26s_53_1_1_U1667/RS_BRIEF_mul_28ns_26s_53_1_1_Multiplier_1_U/B_V_data_1_payload_A[66]_i_2_n_0,
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_RS_BRIEF_U0/mul_28ns_26s_53_1_1_U1667/RS_BRIEF_mul_28ns_26s_53_1_1_Multiplier_1_U/B_V_data_1_payload_A[90]_i_2_n_0,
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_RS_BRIEF_U0/mul_28ns_26s_53_1_1_U1667/RS_BRIEF_mul_28ns_26s_53_1_1_Multiplier_1_U/B_V_data_1_payload_A[95]_i_2_n_0,
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_RS_BRIEF_U0/mul_28ns_26s_53_1_1_U1667/RS_BRIEF_mul_28ns_26s_53_1_1_Multiplier_1_U/B_V_data_1_payload_A[103]_i_3_n_0,
design_1_i/RS_BRIEF_0/U0/grp_process_mdl_fu_86/process_RS_BRIEF_U0/mul_28ns_26s_53_1_1_U1667/RS_BRIEF_mul_28ns_26s_53_1_1_Multiplier_1_U/

B_V_data_1_payload_A[146]_i_3_n_0... and (the first 15 of 15665 listed), GLOBAL_LOGIC0.

d) Implementación full, n=4, **no Heap, 7EV 2022.1/IP 2022.1**

Resource	Utilization	Available	Utilization %
LUT	174693	230400	75.82
LUTRAM	1685	101760	1.66
FF	98588	460800	21.39
BRAM	102	312	32.69
DSP	179	1728	10.36
IO	12	360	3.33
BUFG	16	544	2.94
MMCM	2	8	25.00
PLL	1	16	6.25

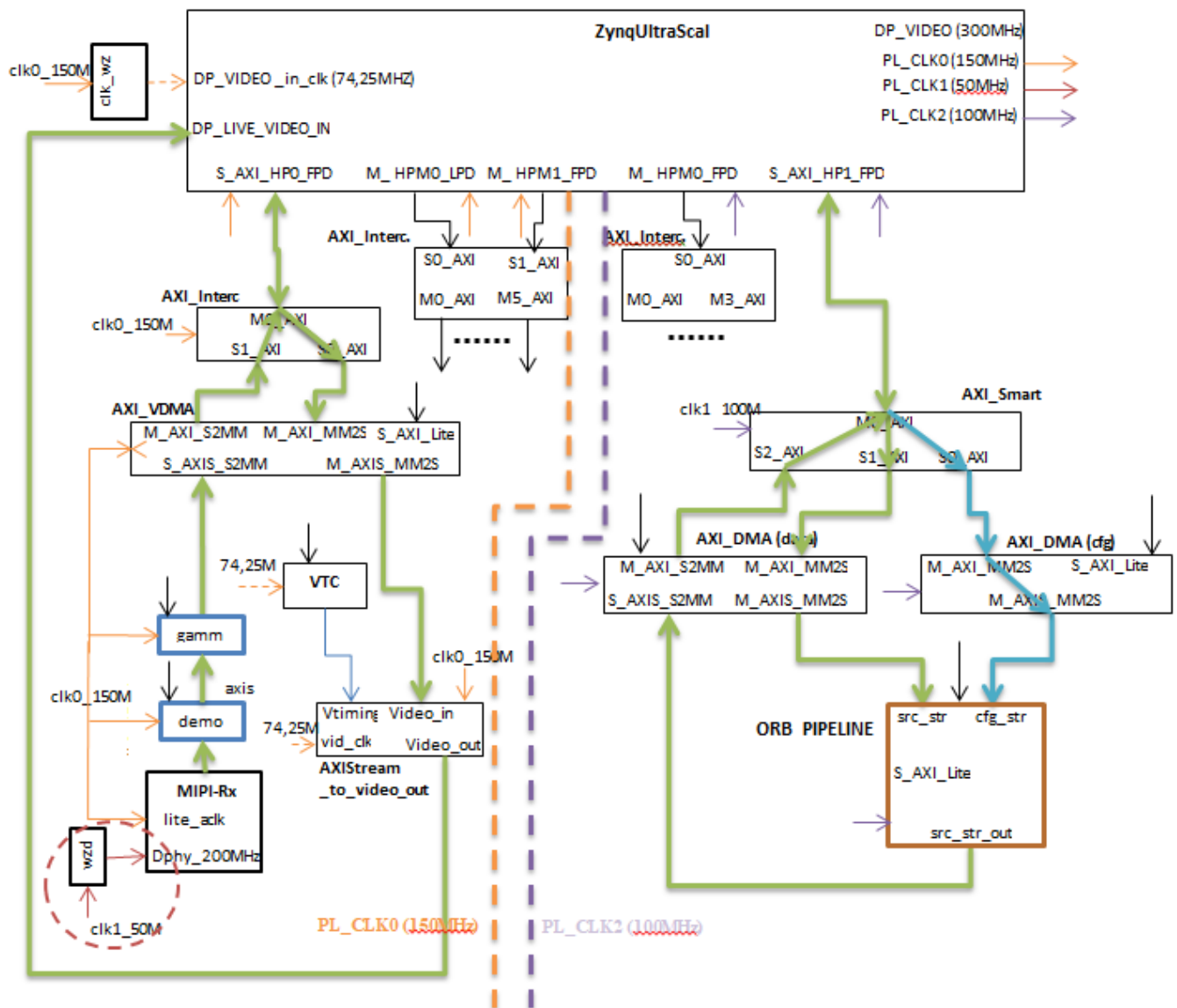
e) Implementación 640x480, n=2, **no Heap, 7EV 2022.1**

XCZU104-MIPI-FULL-PIPELINE\project_n2_640

Resource	Utilization	Available	Utilization %
LUT	165984	230400	72.04
LUTRAM	1464	101760	1.44
FF	88843	460800	19.28
BRAM	92.50	312	29.65
DSP	113	1728	6.54
IO	12	360	3.33
BUFG	14	544	2.57
MMCM	2	8	25.00
PLL	1	16	6.25

- cambiar el Max Burst Size del DMA_data Read al valor 2
- cambiar el pl_clk0 =150MHz cambiar a 100MHz (aparecen algunos problemas de timing: [Timing 38-282] The design failed to meet the timing requirements. Please see the timing summary report for details on the timing violations.)
- reorganizar puertos:
 - M_AXI_HPM0 => AXI_Lites & Control
 - S_AXI_HPC0 => VDMA (150MHz)
 - S_AXI_HPC1 => DMA_data (100MHz)
 - S_AXI_HPC1 => DMA_cfg (100MHz)

9. Arquitectura ORB+MIPI-DP tentativa



AXI_VDMA:
FrameBuffers: 1
• **WriteChannel**
MMDataW: 64
WriteBurstSz: 64
StrDataW: 32
LineBufDepth: 1024
• **ReadChannel**
MMDataW: 64
ReadBurstSz: 64
StrDataW: 32
LineBufDepth: 2048
• **Advanced**
Allow Unaligned Transf

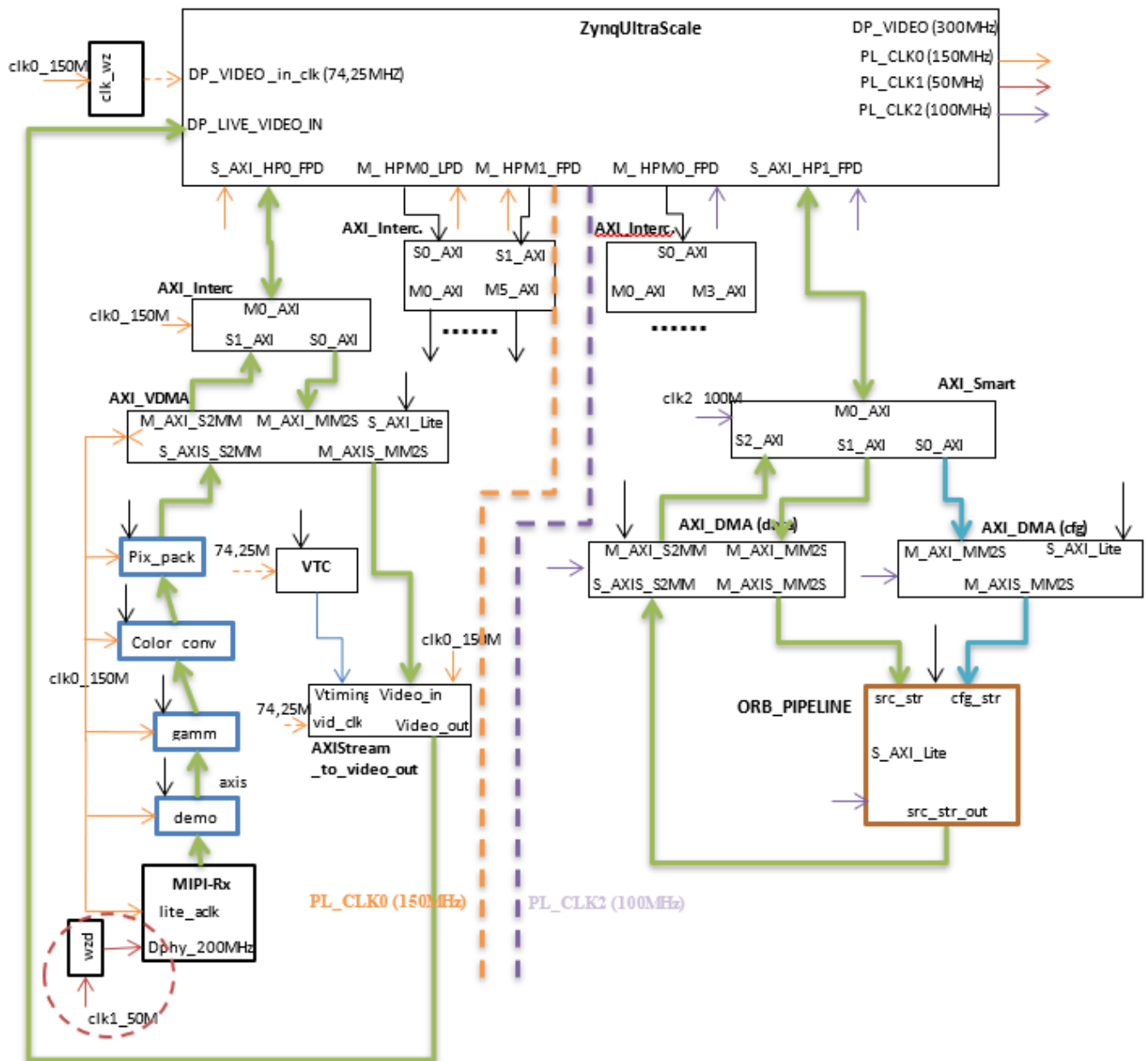
AXI4-StreamtoVideo
FifoDepth: 4096
PixperClock: 1

AXI_DMA_data:
WidthBufLengthReg: 26
AWidth: 32
• **ReadChannel**
MMDataW: 256
StrDataW: 128
MaxBurstSz: 2
• **WriteChannel**
MMDataW: 32
StrDataW: 32
MaxBurstSz: 16

AXI_DMA_cfg:
WidthBufLengthReg: 14
AWidth: 32
• **ReadChannel**
MMDataW: 32
StrDataW: 32
MaxBurstSz: 16

10. Arquitectura ORB+MIPI-DP para imágenes en gris

MIPI incluye la conversión del espacio de color (RGB2YCRCB) y el empaquetamiento de los pixels (Y8)



AXI_VDMA:
FrameBuffers: 1
• **WriteChannel**
MMDataW: 64
WriteBurstSz: 64
StrDataW: 32
LineBufDepth: 1024
• **ReadChannel**
MMDataW: 64
ReadBurstSz: 64
StrDataW: 32
LineBufDepth: 2048
• **Advanced**
Allow Unaligned Transf

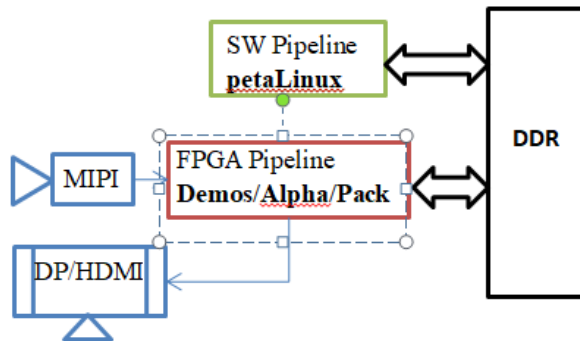
AXI4-StreamtoVideo
FifoDepth: 4096
PixperClock: 1

AXI_DMA_data:
WidthBufLengthReg: 26
AWidth: 32
• **ReadChannel**
MMDataW: 256
StrDataW: 128
MaxBurstSz: 2
• **WriteChannel**
MMDataW: 32
StrDataW: 32
MaxBurstSz: 16

AXI_DMA_cfg:
WidthBufLengthReg: 14
AWidth: 32
• **ReadChannel**
MMDataW: 32
StrDataW: 32
MaxBurstSz: 16

11. Medidas de rendimiento

A) Arquitectura T6.1 PS-PL base



Issue

113

(<https://github.com/embention/MissionComputer/issues/113>)

Captura MIPI imagen BGR / Conversión BGR2Y por Sw / imagen 640x480

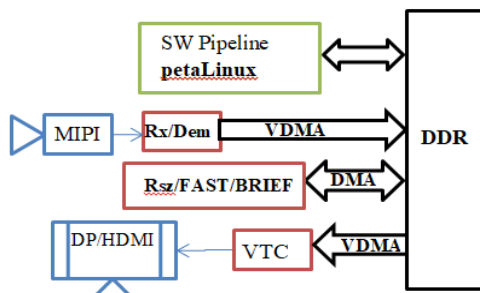
- A: Tiempo transferencia vdma (camara a memoria) = 29,356 milisegundos
- B: Tiempo extraccion de memoria a array = 61,559 milisegundos
- C: Tiempo conversion de array a cv:Mat = 4,273 milisegundos
- Tiempos parciales de ORB SLAM (Solo software):

Tiempos ORB SLAM (PipelineSw)

- Tracking:
 - ORB Extraction: 182,34 ms (desviacion estandar 12,148 ms)
 - Pose Prediction: 48,002 ms (desviacion estandar 3,4374 ms)
 - LM Track: 5,5056 ms (desviacion estandar 0,23367 ms)
 - New KF decision: 0,41386 ms (desviacion estandar 12.148 ms)
- Local Mapping:
 - KF Insertion: 5,34 ms (desviacion estandar 3,4791 ms)
 - MP Culling: 0,99458 ms (desviacion estandar 0,28631 ms)
 - MP Creation: 104,55 ms (desviacion estandar 7,2196 ms)
 - LBA: 395,28 ms (desviacion estandar 0 ms)
 - KF Culling: 0,46543 ms (desviacion estandar 0,45333 ms)

B) Arquitectura T6.2 PS-PL base

[img_i/level1/level2/level3/level4](#) => $T = t_{img} + t_1 + t_2 + t_3 + t_4$



Medidas de rendimiento preliminares para ORB extraction (sin BRIEF, ni Heap)

Módulo/ resolución	640x48 0	1280x72 0
Captura	0.35ms	0.79ms
Rsz+FAST (esc=1.2)	0.33ms	1.58ms

escala	640x48 0	1280x72 0
1.0	0.365	1.618
1.2	0.426	1.07
1.728	0.417	0.417
2.01	0.416	0.413
2.48	0.387	0.429
2.98	0.358	0.420
3.58	0.359	0.411
4.29		
Promedio /escala	0.389	

Tiempo approx. procesamiento de 8 escalas (640x480) => 3,12ms

Es necesario actualizar estos tiempos incluyendo BRIEF+Heap

12. Conclusiones

OBJ1:

- El recurso crítico del pipeline coprocesador es el número de LUTs
- Es posible reducir la ocupación limitando la resolución máxima de la imagen a 640x480
- Una reducción más significativa es posible reduciendo el grado de paralelismo (#pixels procesados en paralelo), aunque en este caso se reducirá también el rendimiento (**por determinar en qué grado**).
- En cualquier caso, **la capacidad mínima para albergar la arquitectura SW-acel (ORB+MIPI-DP) corresponde a una pastilla 7ev.**
- Si además, se quiere añadir coprocesadores para detección y clasificación de objetos, no es posible determinar la capacidad mínima de la pastilla capaz de albergar todo el diseño. **Sería necesario realizar un análisis similar para estos coprocesadores.**

OBJ2:

- Para realizar el desarrollo de la arquitectura SW-acel es necesario contar con **una placa de capacidad superior a la 7ev**, ya que se requerirá integrar módulos de monitorización (ILAs) además del pipeline. Se estima que una placa con **pastilla 9EG** sería suficiente. Los módulos ILA permiten registrar la actividad de las señales en tiempo de ejecución para, posteriormente, depurar el pipeline. Los módulos ILA requieren sobre todo de memoria de bloques BRAM para registrar el estado de las señales durante todos los ciclos que dure la ejecución. Las numeraciones por encima de 7 (EG o EV) requieren Vivado Enterprise, con un coste de \$3595, uso perpetuo pero solo cubre versiones hasta un año desde la compra.

OBJ3:

- La arquitectura propuesta **ORB(Rsz/FAST)+MIPI-DP** funciona correctamente (640x480, 1280x720), con la salvedad de que es necesario realizar un reset del ORB pipeline entre cada transferencia (send/receive). Esto puede hacerse de varias formas:
 - o Por hw: utilizando la señal TLAST del último módulo
 - o Por sw: utilizando un GPIO (AXILite) controlado por sw
- Es necesario realizar los siguientes ajustes:
 - o Añadir un módulo de conversión RGB10 => Y8 en el pipeline (SC). Ahora lo hace el Sw. Opciones:
 - utilizar el módulo VideoProccesing Subsystem
 - crear un módulo HLS**Ver documento 4.Diseño_ImgConditioning.pdf**
 - o Revisar el pipeline ORB para evitar la necesidad de resetear entre transferencias **(SC: no crítico)**
 - o Depurar para escalas >4 **(SC: no crítico)**
 - o Integrar el pipeline Hw en PYNQ
(https://github.com/embention/MissionComputer/tree/a053fa63ab98c539c1a33258da67d67d313b72b1/items/Petalinux_ZUSp/code/zusp_5ev_hw_1v1) (SC: crear proyecto nuevo e incluir la carpeta de srcs para que se genere todo el proyecto)
- Falta estudiar los trade-off ocupación/rendimiento (no crítico)
- **Integración con el pipeline Sw (hablar con John, Revisar el pipeline de tracking)**

Ve documento 5.Integración Acel-SwPipeline